

LISTA DE EXERCÍCIOS

Python para Dados

65 exercícios do fácil ao complexo, em contexto de dados

Bloco 1

Fundamentos, tipos e operadores

8 exercícios · Fácil

Bloco 2

Strings e limpeza de dados

8 exercícios · Fácil a Médio

Bloco 3

Condicionais e repetição

7 exercícios · Médio

Bloco 4

Listas e dicionários

8 exercícios · Médio

Bloco 5

Comprehensions e funções nativas

10 exercícios · Médio a Avançado

Bloco 6

Funções e lambda

5 exercícios · Avançado

Bloco 7

Matrizes e listas de tuplas

5 exercícios · Avançado

Bloco 8

Tratamento de exceções

3 exercícios · Avançado

Bloco 9

Visualização com matplotlib

6 exercícios · Avançado

Bloco 10

Desafios integradores

5 exercícios · Complexo

Como usar esta lista

Os 65 exercícios cobrem tudo que está registrado na sua wiki e nada além disso: fundamentos, operadores, métodos de string, condicionais e laços, listas, dicionários, comprehensions, matrizes, listas de tuplas, funções e lambda, funções nativas (sum, len, round, map, filter, zip, enumerate, sorted, set, any, all, values), tratamento de exceções e matplotlib.

Tudo é resolvível com Python puro e matplotlib. Nenhum exercício exige pandas, numpy ou leitura de arquivos externos: sempre que precisam de dados, eles já vêm embutidos no enunciado, no bloco DADOS, prontos para copiar e colar.

Organização

- Os exercícios estão numerados de 1 a 65 e agrupados em 10 blocos temáticos.
- A dificuldade cresce de forma progressiva: os primeiros blocos firmam a base e os últimos combinam vários conceitos em mini pipelines de ponta a ponta.
- Cada exercício traz o enunciado detalhado, um bloco DADOS quando há dados de entrada, e a lista de conceitos praticados, para você localizar rapidamente o que quer treinar.
- Sugestão: resolva na ordem. Se travar em um bloco avançado, o conceito específico está sendo cobrado em um bloco anterior, é só voltar.

Contexto: os enunciados usam cenários de vendas, milhas, passagens, rotas e logs, o tipo de dado do seu dia a dia, para que a prática já fique parecida com problemas reais.

Bloco 1 — Fundamentos, tipos e operadores

FÁCIL

Variáveis, print com f-string, type(), operadores e round(). O feijão com arroz de qualquer script de dados.

1 Registro de uma venda

Crie quatro variáveis para representar uma venda de passagem: **id_venda** (inteiro, valor 4821), **cliente** (texto, "ana souza"), **valor** (decimal, 389.90) e **pago** (booleano, True). Depois, usando uma única chamada de print() com f-string, exiba a frase: *Venda 4821 de ana souza no valor de R\$ 389.9 (paga: True)*. Os valores mostrados devem vir sempre das variáveis, nunca digitados direto no texto.

Conceitos: variáveis, tipos, print(), f-string

2 Inspeccionando os tipos de um registro

Você recebeu um registro solto de um sistema, com campos de tipos diferentes. Para cada um dos valores abaixo, imprima o próprio valor seguido do seu tipo, usando type(). A saída de cada linha deve ter o formato: *valor -> tipo*. Isso simula a inspeção que se faz antes de decidir como tratar cada coluna.

DADOS

```
id_cliente = 1007
nome = "Bruno Lima"
ticket_medio = 512.75
ativo = True
tags = ["vip", "recorrente"]
```

Conceitos: type(), print()

3 Cálculo de valor líquido

Uma passagem custa **bruto = 1250.00**. Sobre esse valor incide uma taxa de embarque de **78.40** (soma) e um desconto de fidelidade de **12%** aplicado somente sobre o valor bruto (não sobre a taxa). Calcule o valor final a pagar usando apenas operadores aritméticos e imprima o resultado. Deixe claro no print o valor bruto, o desconto em reais e o valor final.

Conceitos: operadores aritméticos, print()

4 Arredondando valores monetários

Um cálculo de rateio gerou os valores **[13.6666, 89.1249, 7.005, 250.9950]**. Para cada valor, imprima a versão arredondada para 2 casas decimais usando round(). Em seguida, imprima também um dos valores arredondado para 0 casas, para comparar o efeito do segundo argumento de round().

Conceitos: round(), listas, for

5 Ticket médio a partir de entrada do usuário

Peça ao usuário, com input(), o valor total faturado no dia e a quantidade de vendas realizadas. Lembre-se de que input() sempre retorna texto: converta o total para decimal (float) e a quantidade para inteiro. Calcule o ticket médio (total dividido pela quantidade), arredonde para 2 casas e imprima com f-string no formato *Ticket médio: R\$ X*.

Conceitos: input(), conversão de tipos, round(), operadores

6 Comparando com a meta

A meta diária de vendas é **meta = 50000.00** e o realizado foi **realizado = 47320.50**. Usando apenas operadores relacionais, crie e imprima três variáveis booleanas: **bateu_meta** (realizado maior ou igual à meta), **ficou_abaixo** (realizado menor que a meta) e **faltou_pouco** (a diferença para a meta é menor que 5000). Imprima as três com rótulos claros.

Conceitos: operadores relacionais, booleanos

7 Elegibilidade para upgrade

Um cliente ganha upgrade de cabine se: for membro do programa de fidelidade **E** (tiver mais de 30000 milhas **OU** ter voado mais de 20 vezes no ano). Dados **fidelidade = True**, **milhas = 24000** e **voos_ano = 22**, escreva uma única expressão usando operadores lógicos (and, or) que resulte em True ou False e imprima se o cliente tem direito ao upgrade.

Conceitos: operadores lógicos, booleanos

8 Acumulando o faturamento

Comece com **total = 0.0**. Você recebe as vendas do dia em quatro momentos: 1200.00, depois 350.50, depois 890.00 e por fim 45.90. Some cada uma ao acumulador usando o operador de atribuição composto (+=), imprimindo o total parcial após cada soma. No fim, imprima o total consolidado.

Conceitos: operadores de atribuição, print()

Bloco 2 — Strings e limpeza de dados

FÁCIL A MÉDIO

Métodos de string para padronizar, extrair e validar texto. É onde 80% do trabalho sujo de dados acontece.

9 Padronizando nomes de clientes

Os nomes chegam do formulário com espaços sobrando e capitalização inconsistente: `[" ana souza ", "BRUNO LIMA", " Carla ", "díEGO ramos "]`. Para cada nome, remova os espaços das pontas, deixe em formato de nome próprio (primeira letra de cada palavra maiúscula) e imprima o resultado. Ao final, todos devem sair limpos, como *Ana Souza*.

DADOS

```
nomes = [" ana souza ", "BRUNO LIMA", " Carla ", "diego ramos "]
```

Conceitos: `strip()`, `title()`, `for`

10 Convertendo valor monetário em número

Um valor veio como texto no padrão brasileiro: `"R$ 1.234,50"`. Você precisa transformá-lo no número decimal 1234.50 para poder somar. Faça a limpeza usando métodos de string em sequência: remova o `"R$ "`, remova o ponto de milhar e troque a vírgula decimal por ponto. Ao final, converta para float e imprima o número resultante mais o seu `type()` para confirmar que virou número.

DADOS

```
bruto = "R$ 1.234,50"
```

Conceitos: `replace()`, `strip()`, conversão float, `type()`

11 Quebrando uma linha de CSV

Você recebeu uma linha de arquivo no formato CSV como uma única string: `"4821;Ana Souza;GRU;389.90;pago"`, onde o separador é ponto e vírgula. Separe a linha em uma lista de campos e imprima cada campo com seu índice, no formato `0: 4821, 1: Ana Souza`, e assim por diante. Depois imprima especificamente o campo de valor (índice 3).

Conceitos: `split()`, listas, indexação

12 Normalizando códigos de aeroporto

Os códigos IATA de origem e destino vieram bagunçados: `[" gru", "Cgh ", "sdu", " GIG "]`. O padrão correto é: sem espaços e todas as letras maiúsculas. Padronize cada código e monte uma nova lista já limpa, imprimindo-a ao final. O resultado esperado é `['GRU', 'CGH', 'SDU', 'GIG']`.

DADOS

```
codigos = [" gru", "Cgh ", "sdu", " GIG "]
```

Conceitos: strip(), upper(), list comprehension

13 Filtrando arquivos por extensão

Uma pasta de ingestão contém os arquivos ["vendas_01.csv", "log.txt", "clientes.CSV", "backup.csv.gz", "rotas.json"]. Você só quer processar arquivos CSV de verdade, ou seja, cujo nome termina exatamente em ".csv" (sem diferenciar maiúsculas de minúsculas, e sem pegar o ".csv.gz"). Percorra a lista e imprima apenas os nomes que atendem à regra.

DADOS

```
arquivos = ["vendas_01.csv", "log.txt", "clientes.CSV", "backup.csv.gz", "rotas.json"]
```

Conceitos: endswith(), lower(), for, if

14 Contando ocorrências em um log

Dada a linha de log "ERROR conexao ERROR timeout WARN retry ERROR falha", descubra quantas vezes a palavra "ERROR" aparece usando o método count(). Depois, descubra a posição (índice) da primeira ocorrência de "WARN" usando find(). Imprima as duas informações com rótulos claros.

DADOS

```
linha = "ERROR conexao ERROR timeout WARN retry ERROR falha"
```

Conceitos: count(), find()

15 Remontando uma linha para exportação

Você tem os campos de um registro já separados em uma lista: ["4821", "Ana Souza", "GRU", "389.90"]. Monte de volta uma única string no formato CSV usando vírgula como separador, através do método join(). Imprima a linha final. Atenção: join() exige que todos os itens sejam texto.

DADOS

```
campos = ["4821", "Ana Souza", "GRU", "389.90"]
```

Conceitos: join(), listas

16 Validação simples de e-mail

Para cada e-mail da lista `["ana@empresa.com", "bruno.com", "carla@", "diego@x.com.br"]`, verifique se ele parece válido com uma regra simples: precisa conter exatamente um "@" e ter pelo menos um "." depois do "@". Imprima cada e-mail seguido de "válido" ou "inválido". Use operadores de string (`in`, `count`) e `split()` para a checagem.

DADOS

```
emails = ["ana@empresa.com", "bruno.com", "carla@", "diego@x.com.br"]
```

Conceitos: `in`, `count()`, `split()`, `if/else`

Bloco 3 — Condicionais e repetição

MÉDIO

if/elif/else e laços for/while para percorrer e classificar registros.

17 Classificando o valor da passagem em faixas

Para cada valor da lista `[89.90, 450.00, 1200.00, 305.50, 2999.99]`, classifique a passagem em faixas: até 200 é "econômica", acima de 200 até 800 é "intermediária", acima de 800 até 2000 é "premium" e acima de 2000 é "executiva". Use `if/elif/else` dentro de um `for` e imprima cada valor com sua classificação.

DADOS

```
valores = [89.90, 450.00, 1200.00, 305.50, 2999.99]
```

Conceitos: `if/elif/else`, `for`

18 Somando com laço explícito

Sem usar a função `sum()`, calcule o faturamento total percorrendo a lista `[1200.0, 350.5, 890.0, 45.9, 780.25]` com um `for` e um acumulador. Imprima o total ao final. O objetivo aqui é praticar o laço manual antes de usar os atalhos prontos.

DADOS

```
vendas = [1200.0, 350.5, 890.0, 45.9, 780.25]
```

Conceitos: `for`, acumulador, operadores de atribuição

19 Paginação simulada com while

Uma API retorna resultados em páginas. Simule a leitura: comece em **pagina = 1** e vá incrementando enquanto **pagina** for menor ou igual a 5. A cada iteração, imprima *Lendo página X...*. Quando o laço terminar, imprima *Ingestão concluída em 5 páginas*. Use **while** e não use **for**.

Conceitos: while, operadores de atribuição

20 Contando vendas aprovadas

Dada a lista de status de pagamento **["pago", "pendente", "pago", "cancelado", "pago", "estornado", "pago"]**, conte quantas vendas estão com status "pago" e quantas não estão. Percorra com **for**, use **if/else** e imprima as duas contagens com rótulos.

DADOS

```
status = ["pago", "pendente", "pago", "cancelado", "pago", "estornado", "pago"]
```

Conceitos: for, if/else, contadores

21 Maior venda sem usar max()

Encontre a maior venda da lista **[780.25, 1200.0, 350.5, 2999.99, 890.0]** percorrendo com **for** e comparando manualmente cada valor a um "maior até agora". Imprima o valor máximo encontrado. Não use a função **max()**.

DADOS

```
vendas = [780.25, 1200.0, 350.5, 2999.99, 890.0]
```

Conceitos: for, if, operadores relacionais

22 Marcando registros para revisão

Percorra os valores **[100.0, -50.0, 320.0, 0.0, 890.0, -12.5]**. Um valor negativo indica erro de estorno mal registrado e um valor igual a zero indica venda de teste. Para cada valor, imprima: *OK* se for positivo, *ERRO: negativo* se for menor que zero e *IGNORAR: teste* se for zero.

DADOS

```
valores = [100.0, -50.0, 320.0, 0.0, 890.0, -12.5]
```

Conceitos: for, if/elif/else

23 Interrompendo e pulando no laço

Você está processando registros até encontrar um marcador de fim. Percorra a lista `["350.5", "vazio", "890.0", "FIM", "45.9"]`. Se o item for "vazio", pule para o próximo com `continue` (não processa). Se for "FIM", encerre o laço com `break`. Para os demais, converta para `float` e some em um acumulador. Ao final, imprima o total somado antes do "FIM" (deve ser 1240.5).

DADOS

```
registros = ["350.5", "vazio", "890.0", "FIM", "45.9"]
```

Conceitos: `for`, `break`, `continue`, `if`

Bloco 4 — Listas e dicionários

MÉDIO

As duas estruturas mais usadas no dia a dia: manipulação, agregação por chave e ordenação.

24 Manipulando uma lista de preços

Parta da lista `precos = [120.0, 350.0, 89.9, 780.0]`. Faça, em ordem: adicione o preço 210.0 ao final; imprima o preço mais caro e o mais barato usando as funções apropriadas; imprima os dois primeiros preços usando fatiamento (`slicing`); e imprima em qual índice está o valor 89.9. Comente com `print` cada resultado.

DADOS

```
precos = [120.0, 350.0, 89.9, 780.0]
```

Conceitos: `append()`, `slicing`, `index()`, `max()`, `min()`

25 Contando vendas por companhia

Dada a lista de companhias de cada venda `["LATAM", "GOL", "LATAM", "AZUL", "GOL", "LATAM", "AZUL"]`, monte um dicionário que conte quantas vendas cada companhia teve. Percorra a lista e, para cada companhia, incremente sua contagem no dict (criando a chave se ainda não existir). Imprima o dicionário final, que deve ser `{'LATAM': 3, 'GOL': 2, 'AZUL': 2}`.

DADOS

```
cias = ["LATAM", "GOL", "LATAM", "AZUL", "GOL", "LATAM", "AZUL"]
```

Conceitos: `dict`, `for`, `if`, contagem por chave

26 Somando o faturamento de um dicionário

Dado o faturamento por rota `{"GRU-SDU": 12500.0, "CGH-BSB": 8300.0, "GIG-CNF": 4100.0}`, calcule o faturamento total somando apenas os valores do dicionário. Use o método `values()` em conjunto com `sum()`. Imprima o total e também o faturamento médio por rota (total dividido pela quantidade de rotas, arredondado para 2 casas).

DADOS

```
fat = {"GRU-SDU": 12500.0, "CGH-BSB": 8300.0, "GIG-CNF": 4100.0}
```

Conceitos: `values()`, `sum()`, `len()`, `round()`

27 Atualizando o cadastro de um cliente

Parta do dicionário `cliente = {"id": 1007, "nome": "Bruno", "milhas": 24000}`. Faça: adicione a chave "cidade" com valor "Belo Horizonte"; atualize "milhas" somando 5000 às milhas atuais; e remova a informação que não deveria estar lá, imaginando que "nome" precise ser corrigido para "Bruno Lima". Imprima o dicionário após cada alteração.

DADOS

```
cliente = {"id": 1007, "nome": "Bruno", "milhas": 24000}
```

Conceitos: `dict`, atribuição por chave

28 Ordenando clientes por milhas

Você tem uma lista de dicionários, cada um representando um cliente. Ordene a lista pela quantidade de milhas, do maior para o menor, usando `sorted()` com uma função lambda na chave `key`. Imprima o resultado ordenado, um cliente por linha, no formato *Nome: X milhas*.

DADOS

```
clientes = [
    {"nome": "Ana", "milhas": 24000},
    {"nome": "Bruno", "milhas": 51000},
    {"nome": "Carla", "milhas": 8000},
    {"nome": "Diego", "milhas": 33000},
]
```

Conceitos: `sorted()`, `lambda`, `dict`

29 Invertendo um dicionário de códigos

Dado o mapeamento de código para cidade {"GRU": "Guarulhos", "SDU": "Rio", "CNF": "Confins"}, crie um novo dicionário invertido, onde a cidade vira a chave e o código vira o valor. Use um dictionary comprehension. Imprima o dicionário invertido.

DADOS

```
aeroportos = {"GRU": "Guarulhos", "SDU": "Rio", "CNF": "Confins"}
```

Conceitos: dict comprehension, items()

30 Consolidando faturamento de dois dias

Você tem o faturamento por rota de dois dias em dicionários separados. Consolide em um único dicionário somando os valores das rotas que aparecem nos dois dias e mantendo as que aparecem em apenas um. Ao final, imprima o dicionário consolidado. A rota "GRU-SDU" deve somar 21000.0.

DADOS

```
dia1 = {"GRU-SDU": 12500.0, "CGH-BSB": 8300.0}
dia2 = {"GRU-SDU": 8500.0, "GIG-CNF": 4100.0}
```

Conceitos: dict, for, if, agregação

31 Top 3 rotas por faturamento

A partir do dicionário de faturamento por rota abaixo, gere um ranking com as 3 rotas de maior faturamento. Transforme os itens do dicionário em uma lista, ordene por valor de forma decrescente com sorted() e lambda, e imprima apenas as 3 primeiras no formato 1. GRU-SDU: R\$ 21000.0.

DADOS

```
fat = {
    "GRU-SDU": 21000.0, "CGH-BSB": 8300.0, "GIG-CNF": 15200.0,
    "SSA-REC": 4100.0, "POA-FLN": 9800.0
}
```

Conceitos: items(), sorted(), lambda, slicing, enumerate()

Bloco 5 — Comprehensions e funções nativas

MÉDIO A AVANÇADO

map, filter, zip, enumerate, set, any, all, sorted e comprehensions: o kit para transformar coleções de forma enxuta.

32 Filtrando vendas acima da média com comprehension

Dada a lista `[120.0, 890.0, 45.9, 350.0, 1200.0, 78.5]`, calcule a média dos valores e, usando uma list comprehension com condição, monte uma nova lista contendo apenas as vendas cujo valor é maior que a média. Imprima a média e a lista filtrada.

DADOS

```
vendas = [120.0, 890.0, 45.9, 350.0, 1200.0, 78.5]
```

Conceitos: list comprehension, sum(), len(), if

33 Aplicando reajuste com comprehension

Todas as tarifas da lista `[100.0, 250.0, 380.0]` sofrerão um reajuste de 8%. Usando list comprehension, gere uma nova lista com os valores reajustados, cada um arredondado para 2 casas decimais. Imprima a lista original e a reajustada, lado a lado, para conferência.

DADOS

```
tarifas = [100.0, 250.0, 380.0]
```

Conceitos: list comprehension, round()

34 Criando um lookup com dict comprehension

Você tem uma lista de códigos de aeroporto `["GRU", "SDU", "CNF", "BSB"]`. Crie, com dictionary comprehension, um dicionário onde cada código é a chave e o valor é o comprimento do código (que aqui será sempre 3, mas o exercício é montar a estrutura). Depois adapte para que o valor seja o código em minúsculas. Imprima os dois resultados.

DADOS

```
codigos = ["GRU", "SDU", "CNF", "BSB"]
```

Conceitos: dict comprehension, len(), lower()

35 Convertendo tipos em massa com map()

Uma coluna numérica veio como texto: `["120", "890", "45", "350"]`. Use map() para converter todos os itens para inteiro de uma vez e transforme o resultado de volta em lista. Imprima a lista convertida e o seu type() para confirmar que agora são inteiros. Some tudo com sum() e imprima o total.

DADOS

```
coluna = ["120", "890", "45", "350"]
```

Conceitos: map(), int, list(), sum()

36 Removendo registros inválidos com filter()

Dada a lista de valores [350.0, -12.0, 0.0, 890.0, -5.5, 120.0], use filter() com uma função lambda para manter apenas os valores estritamente positivos (maiores que zero). Converta o resultado em lista e imprima. Em seguida imprima quantos registros foram descartados, comparando o tamanho antes e depois.

DADOS

```
valores = [350.0, -12.0, 0.0, 890.0, -5.5, 120.0]
```

Conceitos: filter(), lambda, len()

37 Casando colunas com zip()

Você recebeu três colunas separadas: ids [1, 2, 3], clientes ["Ana", "Bruno", "Carla"] e valores [350.0, 890.0, 120.0]. Usando zip(), percorra as três em paralelo e imprima uma linha por registro no formato *Venda 1 | Ana | R\$ 350.0*. Isso reproduz a junção de colunas em registros.

DADOS

```
ids = [1, 2, 3]
clientes = ["Ana", "Bruno", "Carla"]
valores = [350.0, 890.0, 120.0]
```

Conceitos: zip(), for, f-string

38 Numerando linhas com enumerate()

Dada a lista de rotas ["GRU-SDU", "CGH-BSB", "GIG-CNF"], imprima cada rota numerada a partir de 1, no formato *Linha 1: GRU-SDU*. Use enumerate() com o parâmetro start para começar a contagem em 1 em vez de 0.

DADOS

```
rotas = ["GRU-SDU", "CGH-BSB", "GIG-CNF"]
```

Conceitos: enumerate(), start, for

39 Rotas únicas e rotas em comum com set()

Dois dias de operação tiveram as rotas **dia1** = ["GRU-SDU", "CGH-BSB", "GRU-SDU", "GIG-CNF"] e **dia2** = ["GRU-SDU", "POA-FLN", "GIG-CNF"]. Usando `set()`: imprima o conjunto de rotas distintas do dia 1 (removendo a duplicata); imprima quantas rotas distintas existem no total somando os dois dias; e imprima as rotas que aconteceram nos dois dias (interseção).

DADOS

```
dia1 = ["GRU-SDU", "CGH-BSB", "GRU-SDU", "GIG-CNF"]
dia2 = ["GRU-SDU", "POA-FLN", "GIG-CNF"]
```

Conceitos: `set()`, interseção, união, `len()`

40 Checando qualidade dos dados com `any()` e `all()`

Você tem a lista de valores de uma coluna: [120.0, 350.0, 890.0, 45.0]. Responda a duas perguntas com uma única expressão cada: (1) todos os valores são positivos? Use `all()`. (2) existe algum valor acima de 1000? Use `any()`. Imprima as duas respostas com rótulos. Depois teste de novo trocando um valor por -5.0 para ver a resposta mudar.

DADOS

```
valores = [120.0, 350.0, 890.0, 45.0]
```

Conceitos: `all()`, `any()`, comprehension, operadores relacionais

41 Ordenando por chave composta

Você tem uma lista de tuplas (cia, rota, faturamento). Ordene primeiro pela companhia em ordem alfabética e, dentro da mesma companhia, pelo faturamento do maior para o menor. Use `sorted()` com uma lambda que retorne uma tupla de critérios. Imprima o resultado ordenado, uma tupla por linha.

DADOS

```
dados = [
    ("GOL", "CGH-BSB", 8300.0),
    ("LATAM", "GRU-SDU", 21000.0),
    ("GOL", "POA-FLN", 9800.0),
    ("LATAM", "GIG-CNF", 15200.0),
]
```

Conceitos: `sorted()`, lambda, tuplas, ordenação múltipla

Bloco 6 — Funções e lambda

AVANÇADO

Encapsular lógica em funções reutilizáveis, retornar múltiplos valores e usar funções como argumento.

42 Função de limpeza de valor monetário

Escreva uma função **limpar_valor(texto)** que receba uma string no padrão "R\$ 1.234,50" e retorne o número decimal 1234.50. Reaproveite a lógica de remoção de "R\$ ", ponto de milhar e vírgula. Teste a função com pelo menos três valores diferentes, incluindo um sem casas de milhar, e imprima os retornos.

Conceitos: def, return, métodos de string, float

43 Função com estatísticas de uma lista

Escreva uma função **resumo(valores)** que receba uma lista de números e retorne, de uma vez, três informações: o menor valor, o maior valor e a média arredondada para 2 casas. Faça a função retornar os três valores juntos e, na chamada, receba-os em três variáveis separadas. Teste com **[120.0, 890.0, 45.0, 350.0]** e imprima os três resultados.

DADOS

```
valores = [120.0, 890.0, 45.0, 350.0]
```

Conceitos: def, retorno múltiplo, min(), max(), sum(), round()

44 Somando um número variável de vendas

Escreva uma função **faturamento(*vendas)** que aceite qualquer quantidade de valores passados como argumentos separados e retorne a soma de todos. Chame a função de três formas diferentes: com três valores, com um único valor e sem nenhum valor (que deve retornar 0). Imprima cada retorno.

Conceitos: def, *args, sum()

45 Ordenando com uma lambda passada à função

Você tem a lista de tuplas (cliente, milhas) abaixo. Sem escrever um for, ordene a lista pela quantidade de milhas de forma decrescente usando sorted() com uma expressão lambda diretamente no parâmetro key. Imprima o ranking. O objetivo é praticar a lambda anônima em vez de definir uma função nomeada.

DADOS

```
clientes = [("Ana", 24000), ("Bruno", 51000), ("Carla", 8000)]
```

Conceitos: lambda, sorted(), tuplas

46 Pipeline: função que recebe outra função

Escreva uma função **aplicar(coluna, transformacao)** que receba uma lista e uma função de transformação, e devolva uma nova lista com a transformação aplicada a cada item (internamente pode usar map ou comprehension). Teste passando: (1) uma lambda que multiplica por 1.08 (reajuste); (2) uma função nomeada que arredonda para 2 casas. Imprima os dois resultados a partir da lista **[100.0, 250.0, 380.0]**.

DADOS

```
tarifas = [100.0, 250.0, 380.0]
```

Conceitos: def, função como argumento, lambda, map/comprehension

Bloco 7 — Matrizes e listas de tuplas

AVANÇADO

Dados tabulares em Python puro: matrizes (listas de listas) e listas de tuplas, o formato que consultas SQL costumam devolver.

47 Total por linha e por coluna de uma matriz

A matriz abaixo representa vendas por semana (linhas) e por dia útil (colunas). Calcule e imprima: o total de cada semana (soma de cada linha) e o total de cada dia da semana somando todas as semanas (soma de cada coluna). Para as colunas, você pode usar zip() para percorrer os dias em paralelo.

DADOS

```
vendas = [
    [120, 90, 150, 80, 200],
    [100, 130, 170, 60, 210],
    [140, 110, 160, 90, 190],
]
```

Conceitos: listas de listas, for, sum(), zip()

48 Transpondo uma matriz

Dada a matriz **[[1, 2, 3], [4, 5, 6]]** (2 linhas, 3 colunas), gere a matriz transposta (3 linhas, 2 colunas), em que a primeira coluna vira a primeira linha e assim por diante. Faça de duas formas e compare: uma com zip() e outra com list comprehension aninhada. Imprima as duas versões.

DADOS

```
matriz = [[1, 2, 3], [4, 5, 6]]
```

Conceitos: zip(), list comprehension aninhada, matrizes

49 Agregando um resultado de consulta

Uma consulta SQL retornou uma lista de tuplas no formato (companhia, rota, faturamento). Percorra essa lista e monte um dicionário com o faturamento total por companhia. Imprima o dicionário. Em seguida, imprima qual companhia teve o maior faturamento total.

DADOS

```
resultado = [
    ("LATAM", "GRU-SDU", 21000.0),
    ("GOL", "CGH-BSB", 8300.0),
    ("LATAM", "GIG-CNF", 15200.0),
    ("GOL", "POA-FLN", 9800.0),
    ("AZUL", "SSA-REC", 4100.0),
]
```

Conceitos: lista de tuplas, dict, agregação, max com key

50 Convertendo lista de tuplas em dicionário

Dada a lista de tuplas (codigo, cidade) [("GRU", "Guarulhos"), ("SDU", "Rio"), ("CNF", "Confins)], converta-a diretamente em um dicionário onde o código é a chave e a cidade é o valor. Faça com a função dict() e também com dict comprehension. Depois consulte e imprima a cidade correspondente ao código "CNF".

DADOS

```
pares = [("GRU", "Guarulhos"), ("SDU", "Rio"), ("CNF", "Confins")]
```

Conceitos: lista de tuplas, dict(), dict comprehension

51 Tabela dinâmica manual

Você tem uma lista de tuplas (mes, canal, valor) com vendas por canal e por mês. Monte uma estrutura que permita consultar o total de cada canal em cada mês, por exemplo um dicionário cuja chave é a tupla (mes, canal). Percorra os dados, some os valores por combinação e imprima o resultado de forma organizada, uma combinação por linha.

DADOS

```
dados = [
    ("jan", "site", 5000.0), ("jan", "app", 3000.0),
    ("jan", "site", 2500.0), ("fev", "site", 4000.0),
    ("fev", "app", 3500.0), ("fev", "app", 1500.0),
]
```

Conceitos: lista de tuplas, dict com chave composta, agregação

Bloco 8 — Tratamento de exceções

AVANÇADO

Dados reais têm sujeira. try/except mantém o pipeline de pé mesmo com registros quebrados.

52 Convertendo uma coluna suja sem quebrar

A coluna numérica veio com lixo no meio: `["120", "abc", "350", "", "890", "12x"]`. Percorra a lista tentando converter cada item para inteiro dentro de um try/except. Some os que derem certo em um acumulador e conte quantos falharam. Ao final imprima o total válido e a quantidade de registros descartados. O total válido deve ser 1360.

DADOS

```
coluna = ["120", "abc", "350", "", "890", "12x"]
```

Conceitos: try/except, ValueError, for, contadores

53 Ticket médio protegido contra divisão por zero

Escreva uma função `ticket_medio(total, qtd)` que retorne o total dividido pela quantidade. Proteja o cálculo com try/except para o caso de a quantidade ser zero, retornando 0.0 nessa situação em vez de deixar o programa quebrar. Teste com (10000.0, 40) e com (10000.0, 0), imprimindo os dois retornos.

Conceitos: def, try/except, ZeroDivisionError, return

54 Parsing robusto de registros

Cada registro é uma string "id;valor", mas alguns vêm malformados: `["1;350.0", "2;abc", "semponto", "4;890.0", "5;"]`. Escreva um laço que, para cada registro, tente separar em id e valor e converter o valor para float. Use try/except para capturar tanto erros de separação quanto de conversão. Os registros válidos vão para uma lista de tuplas (id, valor); os inválidos vão para uma lista de erros. Imprima as duas listas ao final.

DADOS

```
registros = ["1;350.0", "2;abc", "semponto", "4;890.0", "5;"]
```

Conceitos: try/except, split(), float, listas de tuplas

Bloco 9 — Visualização com matplotlib

AVANÇADO

Transformar números em gráficos com matplotlib.pyplot: linha, barra, histograma, pizza e dispersão.

55 Série temporal de faturamento

Com os dados de faturamento diário abaixo, crie um gráfico de linha usando plt.plot(). Coloque os dias no eixo X e o faturamento no eixo Y. Adicione título "Faturamento diário", rótulos nos dois eixos, ative a grade com plt.grid() e exiba com plt.show(). Use marcadores nos pontos para deixar cada dia visível.

DADOS

```
dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]  
faturamento = [12000, 9500, 15000, 11000, 18000]
```

Conceitos: plt.plot(), title, xlabel, ylabel, grid, show

56 Comparando companhias em barras

Crie um gráfico de barras com plt.bar() comparando o faturamento total de cada companhia. As companhias vão no eixo X e o faturamento no eixo Y. Dê um título, rotule os eixos e exiba o gráfico. Como desafio, ordene as companhias do maior para o menor faturamento antes de plotar, usando sorted() com lambda.

DADOS

```
cias = ["LATAM", "GOL", "AZUL"]  
faturamento = [36200, 18100, 4100]
```

Conceitos: plt.bar(), sorted(), lambda, title, labels

57 Distribuição de preços com histograma

Você tem os preços de várias passagens vendidas. Crie um histograma com plt.hist() para visualizar em quais faixas de preço as vendas se concentram. Use 5 faixas (bins). Coloque título "Distribuição de preços", rótulo "Preço (R\$)" no eixo X e "Frequência" no eixo Y, e exiba.

DADOS

```
precos = [120, 130, 150, 200, 210, 220, 400, 410, 420, 800, 90, 110, 205, 215, 395]
```

Conceitos: plt.hist(), bins, title, xlabel, ylabel

58 Participação por canal em pizza

Crie um gráfico de pizza com plt.pie() mostrando a participação de cada canal de venda no total. Exiba os rótulos de cada fatia e a porcentagem em cada uma (parâmetro autopct). Dê um título ao gráfico e exiba. Some os valores antes para conferir se batem 100%.

DADOS

```
canais = ["Site", "App", "Call Center", "Parceiros"]  
vendas = [5200, 3800, 1200, 900]
```

Conceitos: plt.pie(), autopct, labels, title

59 Preço x distância em dispersão

Investigue a relação entre a distância da rota (km) e o preço da passagem com um gráfico de dispersão (plt.scatter()). Distância no eixo X, preço no eixo Y. Adicione título, rótulos nos eixos e grade. Ative a legenda com plt.legend() rotulando a série como "Rotas nacionais". Exiba o gráfico.

DADOS

```
distancia = [360, 430, 1100, 850, 2100, 600, 1500]  
preco = [150, 180, 420, 350, 780, 210, 560]
```

Conceitos: plt.scatter(), label, legend, grid, title

60 Duas séries e exportação para arquivo

Plote no mesmo gráfico duas linhas: o faturamento de 2024 e o de 2025 ao longo dos meses. Cada linha deve ter seu próprio rótulo (label) para aparecer na legenda. Adicione título, rótulos dos eixos, grade e legenda. Em vez de exibir na tela, salve o gráfico em um arquivo PNG com plt.savefig(), usando dpi adequado para ficar nítido. Ao final imprima o caminho do arquivo salvo.

DADOS

```
meses = ["Jan", "Fev", "Mar", "Abr"]  
f2024 = [30000, 28000, 35000, 40000]  
f2025 = [42000, 39000, 47000, 51000]
```

Conceitos: plt.plot() múltiplo, legend, savefig, dpi

Bloco 10 — Desafios integradores

COMPLEXO

Cada desafio combina limpeza, estruturas, agregação, funções e visualização em um mini pipeline de ponta a ponta.

61 Do texto sujo ao ranking de rotas

Você recebeu uma lista de registros em texto, um por venda, no formato "rota|valor", com valores no padrão brasileiro e alguns registros corrompidos. Construa um pipeline completo: (1) percorra os registros tratando erros com try/except; (2) limpe e converta o valor para float; (3) agregue o faturamento total por rota em um dicionário; (4) gere um ranking decrescente das rotas com sorted() e lambda; (5) imprima o ranking numerado com enumerate() no formato 1. GRU-SDU: R\$ 21500.00. Registros inválidos devem ser contados e reportados no final.

DADOS

```
registros = [  
    "GRU-SDU|R$ 12.500,00",  
    "CGH-BSB|R$ 8.300,00",  
    "GRU-SDU|R$ 9.000,00",  
    "corrompido",  
    "GIG-CNF|R$ 4.100,00",  
    "GRU-SDU|abc",  
    "GIG-CNF|R$ 3.200,00",  
]
```

Conceitos: try/except, string cleaning, dict, sorted, lambda, enumerate

62 Relatório de desempenho por companhia

Partindo de um resultado de consulta (lista de tuplas com companhia, rota e faturamento), produza um relatório: (1) faturamento total e número de rotas por companhia, guardados em um dicionário; (2) o ticket médio por rota de cada companhia (total dividido pelo número de rotas, arredondado); (3) impressão organizada, uma companhia por linha, ordenada pelo faturamento total decrescente; (4) por fim, um gráfico de barras com plt.bar() do faturamento total por companhia, com título, rótulos e grade. Encapsule o cálculo das métricas em pelo menos uma função.

DADOS

```

resultado = [
    ("LATAM", "GRU-SDU", 21000.0),
    ("GOL", "CGH-BSB", 8300.0),
    ("LATAM", "GIG-CNF", 15200.0),
    ("GOL", "POA-FLN", 9800.0),
    ("AZUL", "SSA-REC", 4100.0),
    ("LATAM", "SDU-CGH", 6000.0),
]

```

Conceitos: lista de tuplas, dict, def, round, sorted, matplotlib

63 Deduplicação e checagem de qualidade

Você tem uma lista de tuplas (id_venda, cliente, valor), mas há linhas duplicadas (mesmo id) e problemas de qualidade. Faça: (1) remova duplicatas mantendo apenas a primeira ocorrência de cada id_venda, usando um set() de controle; (2) verifique com all() se todos os valores restantes são positivos e com any() se existe algum acima de 5000; (3) conte quantos ids únicos sobraram; (4) imprima um pequeno relatório de qualidade com essas informações e a lista final deduplicada.

DADOS

```

vendas = [
    (1, "Ana", 350.0),
    (2, "Bruno", 890.0),
    (1, "Ana", 350.0),
    (3, "Carla", 6200.0),
    (2, "Bruno", 890.0),
    (4, "Diego", 120.0),
]

```

Conceitos: set(), lista de tuplas, all(), any(), len()

64 Mini ETL de vendas por rota

Implemente um pequeno ETL de ponta a ponta a partir de linhas de texto no formato "data;rota;valor", com valores no padrão americano (ponto decimal) e alguns registros com valor vazio ou inválido. Etapas: (1) EXTRACT: parta da lista de strings; (2) TRANSFORM: separe os campos, converta o valor para float dentro de try/except, descarte registros inválidos e normalize a rota para maiúsculas sem espaços; (3) LOAD: agregue o faturamento total por rota em um dicionário e calcule o total geral; (4) SAÍDA: imprima o total por rota ordenado do maior para o menor e o total geral; (5) VISUALIZAÇÃO: gere um gráfico de barras do faturamento por rota e salve em PNG com plt.savefig(). Organize cada etapa em uma função separada.

DADOS

```
linhas = [  
    "2025-01-05; gru-sdu ;350.00",  
    "2025-01-05;CGH-BSB;890.50",  
    "2025-01-06;gru-sdu;;",  
    "2025-01-06;GIG-CNF;120.00",  
    "2025-01-07;cgh-bsb;abc",  
    "2025-01-07;GRU-SDU;780.25",  
]
```

Conceitos: funções, try/except, string cleaning, dict, sorted, matplotlib

65 Painel resumido de indicadores

Consolide tudo em um painel textual de indicadores a partir de uma lista de tuplas (rota, canal, valor). Calcule e imprima: (1) faturamento total geral; (2) faturamento por canal, em porcentagem do total, arredondado para 1 casa; (3) a rota de maior faturamento; (4) o número de rotas distintas e de canais distintos usando set(); (5) quantos registros estão acima do ticket médio geral. Reaproveite funções nativas (sum, len, sorted, set, round) sempre que possível e monte a impressão final como um pequeno dashboard organizado por seções.

DADOS

```
dados = [  
    ("GRU-SDU", "site", 12500.0),  
    ("CGH-BSB", "app", 8300.0),  
    ("GRU-SDU", "app", 9000.0),  
    ("GIG-CNF", "site", 4100.0),  
    ("POA-FLN", "parceiros", 9800.0),  
    ("GIG-CNF", "app", 3200.0),  
    ("GRU-SDU", "site", 6000.0),  
]
```

Conceitos: sum, len, set, sorted, round, dict, agregação